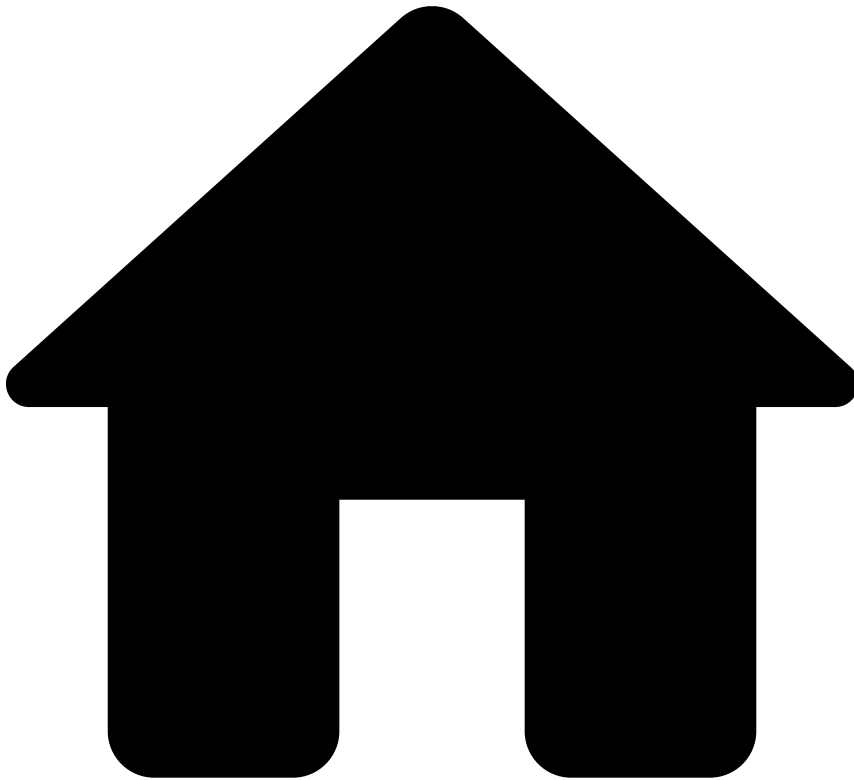


•

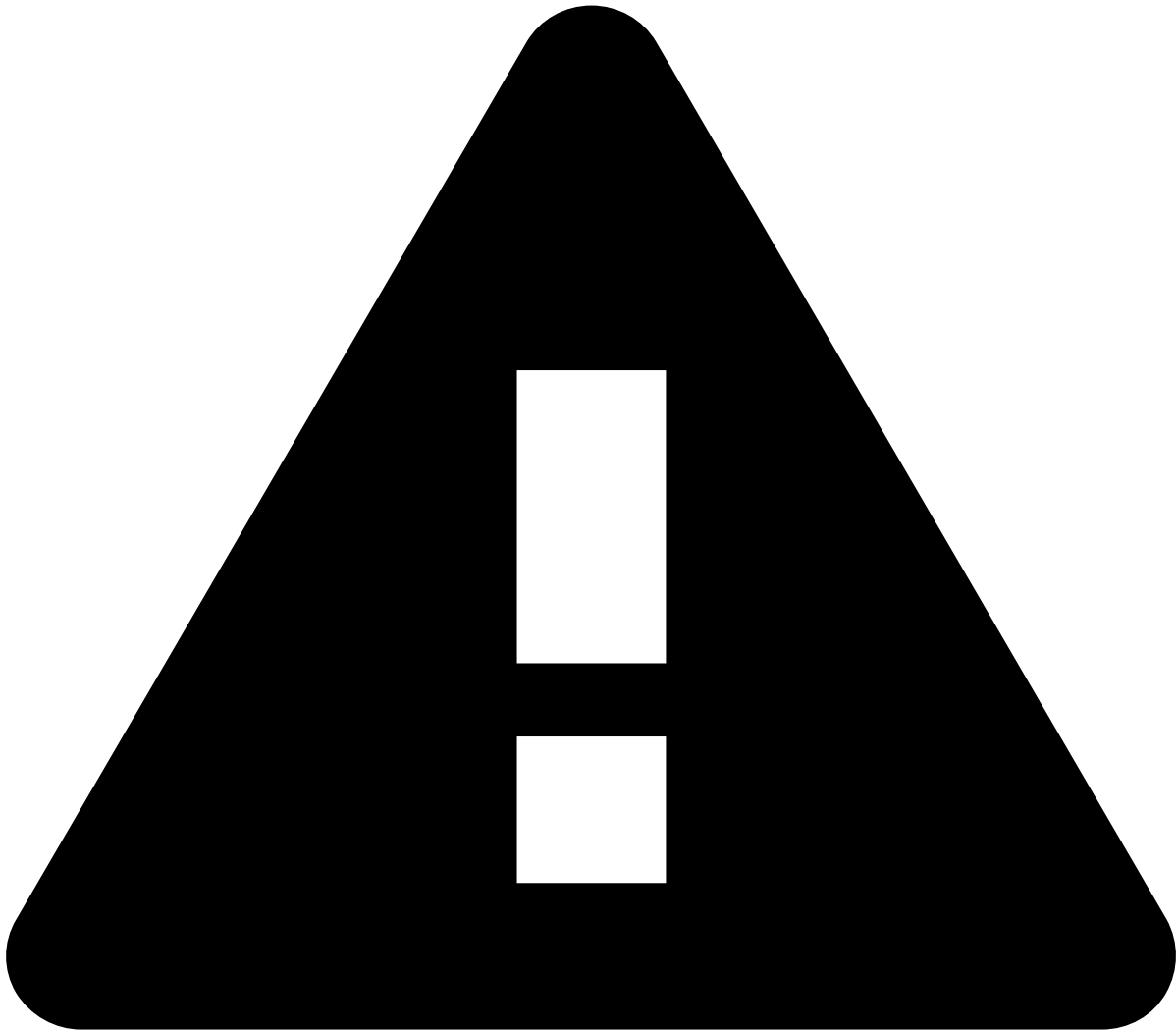


- [Operational Work](#)
- [Data Scientist & Fraud Strategist](#)
- [Work with Rules](#)
- Publish Rules to Runtime

On this page

Publish Rules to Runtime

Was this page helpful? 



warning

If you are working on a **pre-prod environment**, make sure you [migrate resources between environments](#) first, i.e. export resources from pre-prod and import them into the production environment.

A workflow consists of processing steps (workflow services) and links between them. A workflow service is a Pulse component that maps to Pulse resources such as plans and rules, and performs their functionality in real time.

These services are configured using static versions of the resources developed in the **Data Science** tab (i.e. snapshots). This allows you to continue evolving resources in the **Data Science** tab while the workflow processes transactions with your current snapshots. Once the work in the data science tab is fully developed, you need to create new snapshots and update the workflow.

To get the best out of rules you need to understand the mapping between resources and the runtime workflow. Learn how to activate your resources in the Pulse runtime workflow to process transactions in real time.

Workflow mapping📄📄

The following diagram illustrates how Pulse concepts translate from resources to workflow services in runtime:

Standard Workflow📄📄

Transaction Fraud for Banking (TFB) includes a standard workflow for each API.

Workflow settings

Use the **Workflow** tab to open each API's workflow.

- **Schema:** Workflow's input schema, which contains the API's available fields.
- **Outcomes:** Use this option to configure internal fields that will be used to store the outcome of rules and models.
For instance:
 - `decision` : Standard field that is updated when a rule is triggered with the value defined by the **Mark as** option (see the example provided in the [Rules](#) page).
- **Outcome Combinator:** This field allows you to combine different scores in case you have more than one element in the workflow contributing to the score (i.e. you can have both models and rules contributing to the score).



info

Outcome fields: The outcome fields are combined with the input schema fields in a new schema named *Extended API Schema version*.

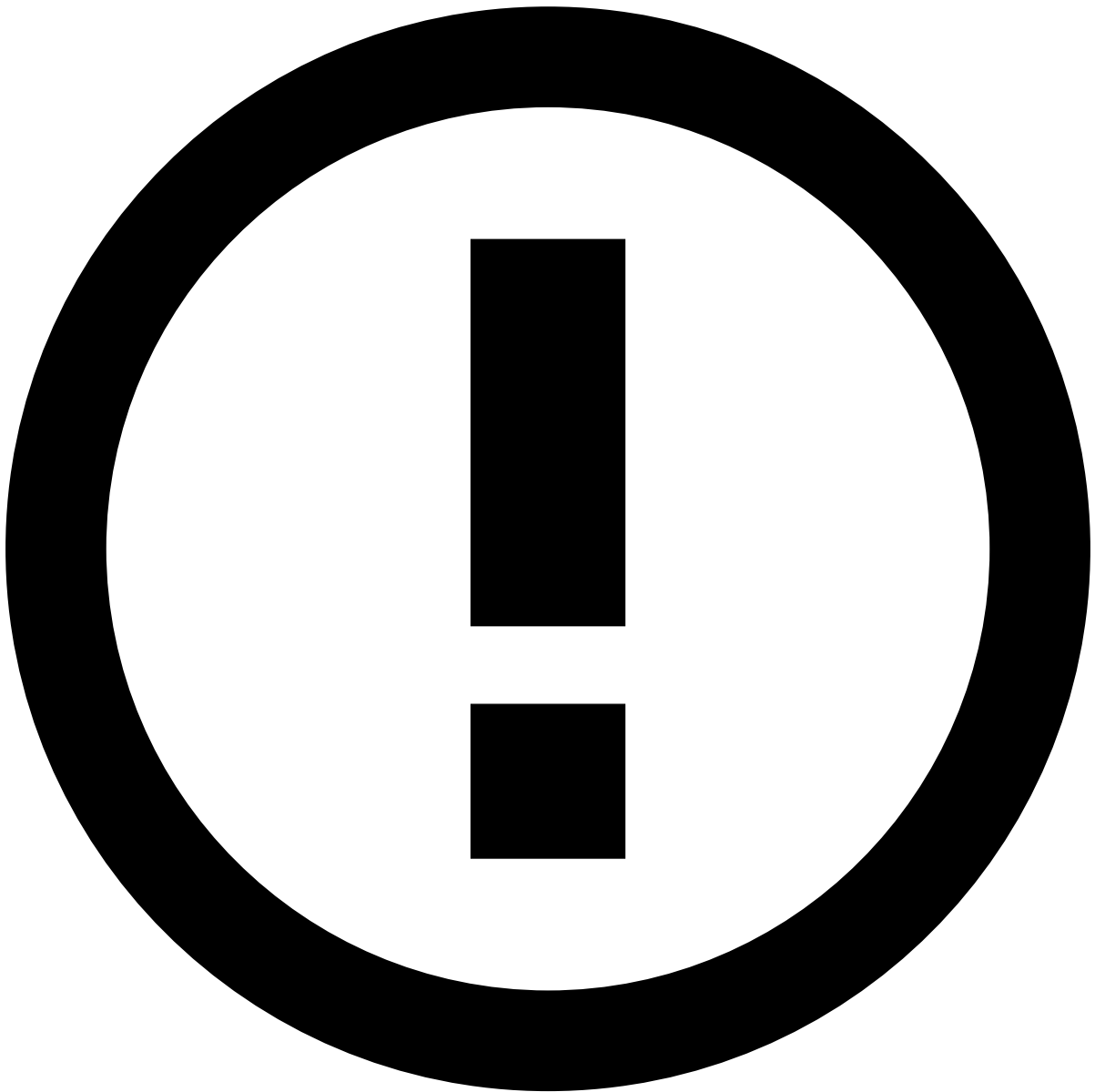
Workflow diagram

For your convenience, the following diagram belongs to Transfers API's workflow, which includes the majority of services among the remaining APIs.

- **Input Adapter:** Receives transactions provided by your system using the schema available in the TFB API. The output of the service is an *Extended API Schema version* with the internal fields defined in the [Outcomes](#) option.
- **Plans:** Computes the fields defined in the plan. Typical examples include profiles such as count of the number of transactions in the last 3 minutes. The input and output schemas are configured when you create a plan execution in the **Data Science** tab, and typically have the following names:
 - **Input Schema:** *Extended API Schema version*
 - **Output Schema:** *Workflow Schema version*
- **Rules:** Applies rules selected by rules projects' snapshots. The input schema of the rules service is the output schema of the last plan that precedes rule services (e.g. *Workflow Schema version*).
- **Custom Workflow Elements:** The standard workflow provides the following services:
 - **Expired Metrics Service:** Service related to metrics such as "how many transactions in the last x days". Due to limitations of real-time metric windows, each of these metrics is composed of a scheduled metric and a shorter real-time metric. They overlap to ensure events are not undercounted. If the job to calculate the scheduled metric fails, this service marks the scheduled metric as expired. In this case, Strong Customer Authentication is enforced as it's unclear whether transactions will be undercounted. This ensures the system is compliant with PSD2 requirements.
 - **Workflow Monitoring Service:** Collects metrics regarding how many events passed through the workflow. This service creates a different bucket for each combination of values of all the configured event fields, outcomes and triggered actions.
 - **List Management Service:** Service that checks rule actions generated on each rules service and performs list item transitions as defined in the [List Management Service](#).
 - **Alert Storage Service:** Sends a transaction to Case Manager when a rule is triggered with the `alert` Action.
 - **Outcome Priority Enforcer:** Custom service that looks at the outcome value of each triggered rule (i.e. high, low, and medium) to enforce the final decision outcome. This outcome is written to the `enforced_risk_level` field. Learn more about rule [outcomes](#).
 - **Revelock (i.e. Digital Trust) Enricher Service:** Service that collects data related to device, network and behavior. It returns and processes this raw data to give data scientists indicators of fraud patterns such as account takeover and its threats (phishing, malware, suspicious network activity, among others). Data scientists can create [rules](#), and build [models](#) to detect and prevent these fraud scenarios. Note that the **Revelock (i.e. Digital Trust) Enricher** service receives only `revelock` events, while the **Disabled Revelock (i.e. Digital Trust) Enricher** service receives only `non-revelock` events.
 - **Sanctions Enricher Service:** Service that handles the integration with third-party sanctions screening providers and collects screening results. This is only relevant if you have Feedzai's Watchlist Transaction Screening solution enabled. Otherwise, it is disabled, even if visible in the interface.
 - **Data Migration Service:** Service that stores received events into accessible storage in different environments. This service's purpose is to help data scientists conduct research on runtime events. It also converts received events into the Parquet format, thus organizing the information more efficiently.

Publish changes in workflow services

1. In the **Workflow** tab, edit each operator in the **diagram** of the *Cards Workflow* using the button:
 1. Select the resource and snapshots' latest version. For instance, if in the previous steps you created snapshots for plans and / or rules, use those snapshots in the workflow respective services (plan / rules project, respectively).
 2. **Save** your changes.
 3. **Save** and exit the workflow.
2. Publish the workflow using the **Publish (Live)** button.
3. Apply your changes to complete the process using the **Rolling Publish** button.
 1. Never skip recovery.
 2. Typically, changing schemas, rules, or plans does not require restarting the application process. If in doubt, reach out to Feedzai for help.



info

Feedzai recommends that you use the **Rolling Publish** option. This option publishes application changes on one Pulse replica at a time, thus reducing the downtime, as there may be other online replicas still able to handle events.

If during the creation of your rule, you used the shadow (NO-OP) mode, go back to editing your rule and take the following steps:

1. Remove `NO-OP` from **Actions** and add `Alert` (if needed).
2. **Mark As** `decline` (or `approve/review`), according to the definition of your rule.
3. **Save** the rule.
4. Create a new [rule snapshot](#).
5. Repeat the steps to publish these changes in the workflow services.

Next steps📄

Now that you know how to apply your rules in runtime to process transactions in real time, learn about [testing your work via API](#).

